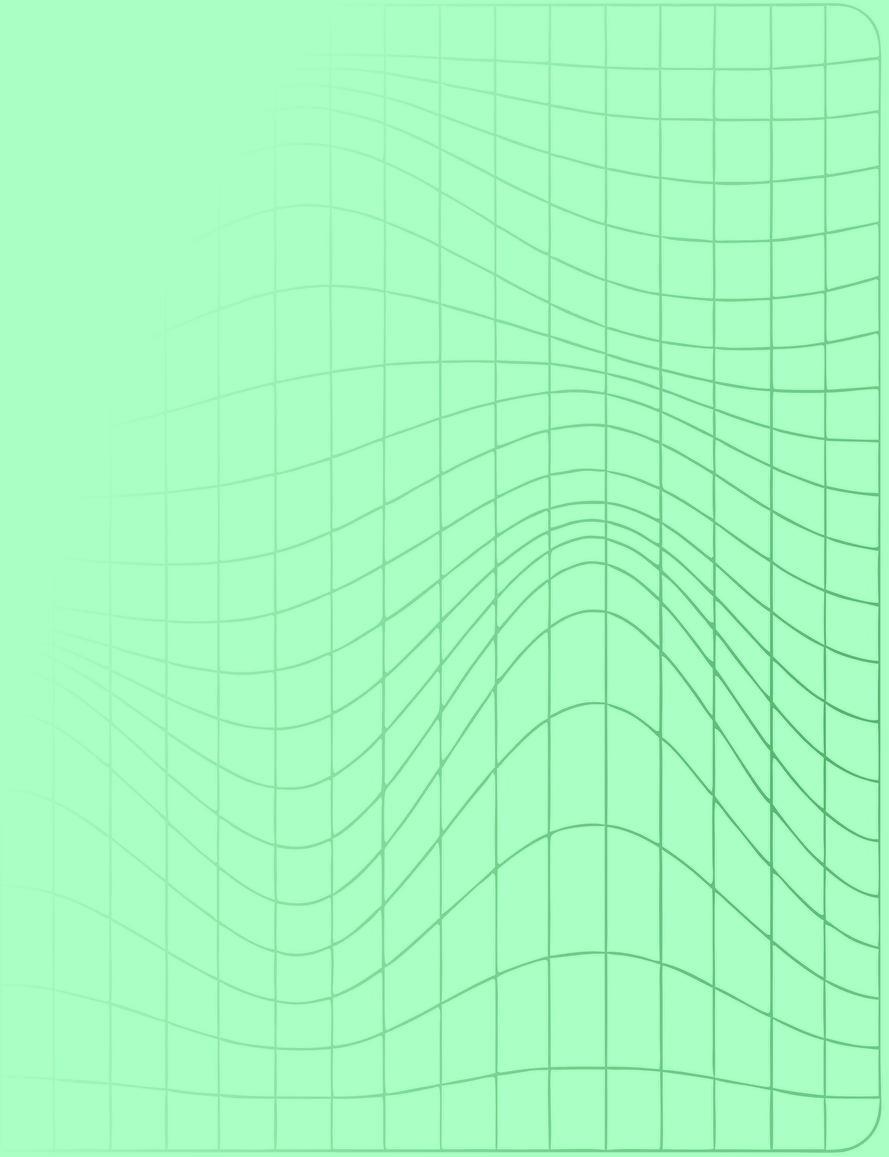
The Immune System for Software

Lessons from biology for the future of software reliability



Phoebe

Phoebe’s agent is saving thousands of hours of manual work, and improving MTTR by 20%. . .

But we think the next era of agents need to be built in a different way.

1 Why software reliability should be more like an immune system

2 The importance of data retrieval

3 Which parts you should own in-house?

Despite new tools and more data, reactive manual work keeps scaling

PREVENTATIVE

Premeditated defences

- ✓ Testing
- ✓ Feature flags
- ✓ Load balancing
- ✓ Auto-scaling
- ✓ Circuit breakers
- ✓ Rate limiting
- ✓ Resets & retries

REACTIVE

Learned, targeted responses

- ⚠ Observability + alerting
- ⚠ Manual diagnosis
- ⚠ Manual resolution
- ⚠ Manual learning

Our bodies’ immune systems can adapt to new types of illness

PREVENTATIVE: INNATE IMMUNITY

Fast and general, such as skin barrier, inflammation, fever response.

Buys time for the adaptive system.

REACTIVE: ADAPTIVE IMMUNITY

Ubiquitous detection, always on, anywhere in the body.

Every encounter makes the system smarter.

To fix novel software issues preemptively, we need capabilities beyond “AI SRE”

AI SRE IS STILL REACTIVE

- ⊗ Alert triage chatbots
- ⊗ Conversational incident interfaces
- ⊗ Automated runbook execution
- ⊗ Post-incident summarisation
- ⊗ Auto-generated postmortems

WHAT AN IMMUNE SYSTEM NEEDS

- ✓ Continuous pre-incident monitoring
- ✓ Predictive failure detection
- ✓ Cross-source evidence correlation
- ✓ Learned system behaviour models
- ✓ Graduated autonomous action

Today’s detection isn’t effectively using early warning signals

RESOURCE & INFRA

- Memory leaks / growth trends
- CPU / I/O saturation trends
- Disk fill rate, inode exhaustion
- Network error rate rises

APPLICATION METRICS

- Latency tail slow growth
- Error-rate 'fuzz' (low 4xx/5xx)
- Queue depth / backlog growth
- Retry storm patterns

DEPLOYMENT SIGNALS

- Config drift (infra or flags)
- Version-to-error correlations
- Test failures or flakiness

LOG & TRACE ANOMALIES

- New error signatures, even if rare
- Increasing variance in trace spans
- Growing retries / CB half-open

USER BEHAVIOUR

- Traffic stress bottlenecks
- Unusual user flows
- Transaction success rate drift

DEPENDENCY HEALTH

- Upstream dependency flakiness
- Service dependency stress points

The first era of “AI SRE” is held back by key architectural weaknesses

1

It’s dependent on monitoring built for humans

2

It takes data outside your environment, therefore has access to fewer data sources

3

The token consumption is too costly to run constantly

1 Why software reliability should be more like an immune system

2 The importance of data retrieval

3 Which parts you should own in-house?

Your AI is constrained by data challenges that MCP can’t solve

DATA SECURITY

Each agent is wired directly to each data source, with all or nothing access and agents directly holding API credentials.

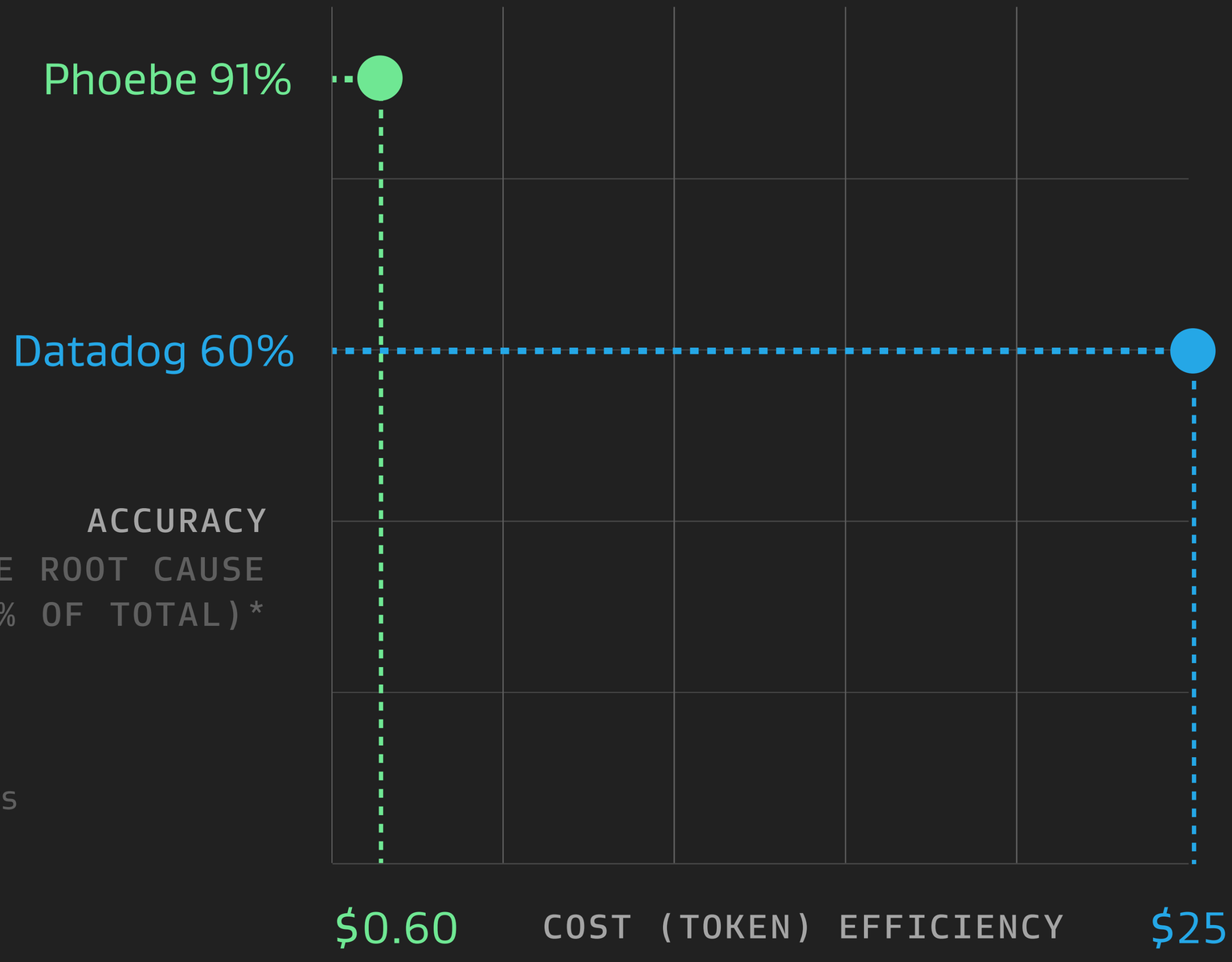
RETRIEVAL ACCURACY

Agents have inconsistent retrieval accuracy from large scale, siloed data sources due to unfamiliar schemas and query languages.

TOKEN EFFICIENCY

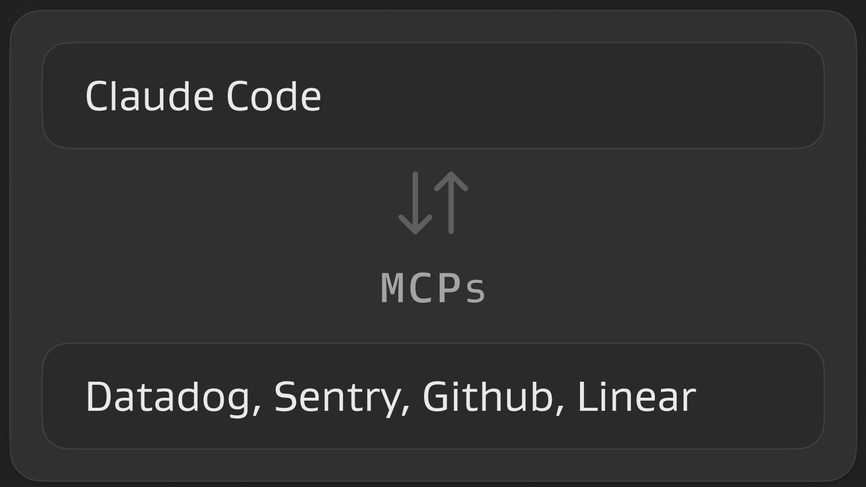
Agents often burn tokens with broad queries, and flood context windows trying to correlate evidence from multiple MCPs.

Data access and retrieval have a big impact on performance

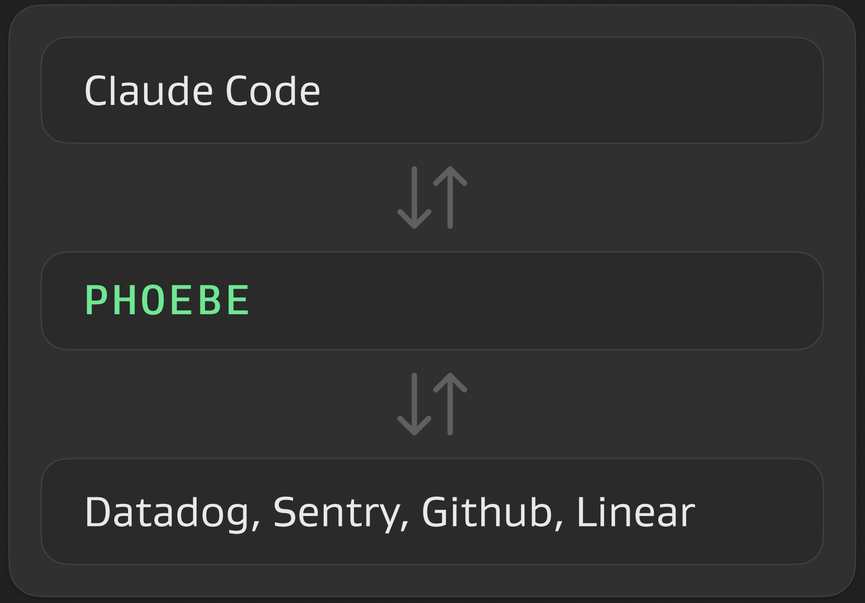


*% of AI investigations on Datadog alerts matching human analysis of root cause
[full + partial diagnoses] / [total]

MCP vs Phoebe using the same LLM, agent framework and data sources



VS



10%
more accurate

86%
fewer tokens

46%
lower cost

48%
faster

How does data retrieval make so much difference to agent performance?

CONTEXT LIMITS

Agents often get too much irrelevant data — burning tokens and confusing reasoning.

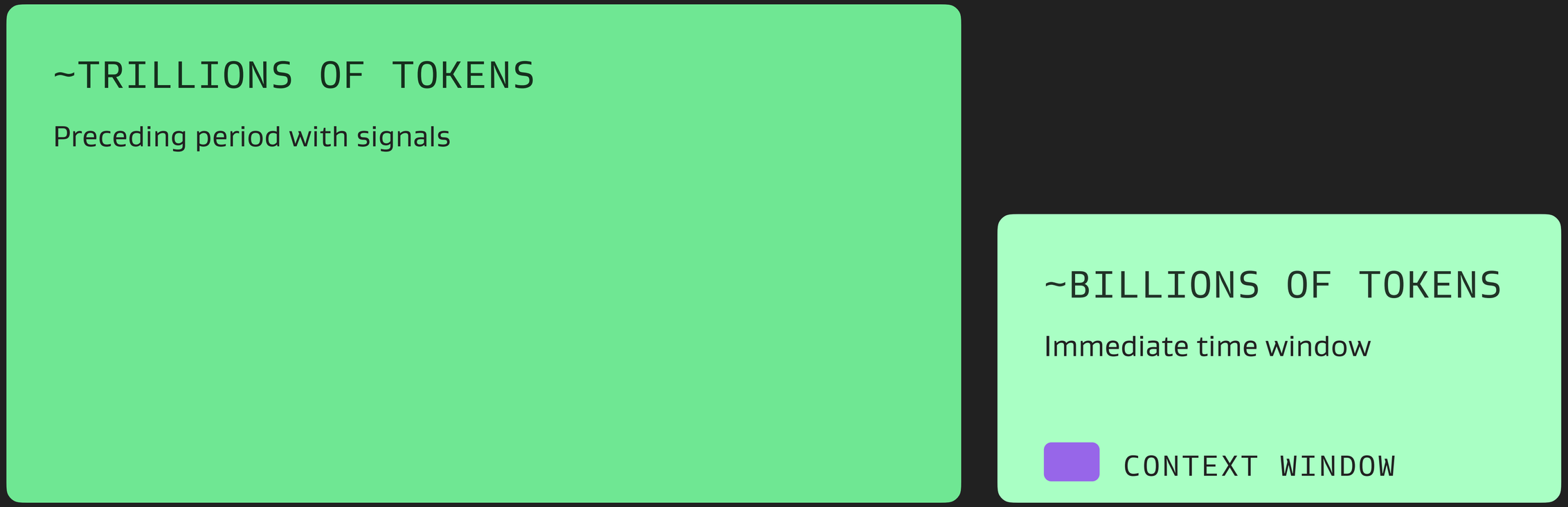
REASONING SILOS

Naive retrieval from siloed systems means the agent is weaker at correlating signals.

MEMORY GAPS

An agent with no memory of your system's behaviour often doesn’t know what to search for.

The time period for incident evidence often has multiple TBs of data (trillions of tokens)



Where retrieval fails at large scale

Token burn from broad queries

The agent doesn’t know which parameters to query for.

Context flooding from multiple MCPs

Six different tools return unstructured blobs that the agent combines in a constrained context window.

Source unfamiliarity

Each source has its own query language and data schemas unfamiliar to the agent.

No cross-source correlation

The config change at 14:00 and errors at 14:12 are in different systems, retrieved independently.

What good retrieval looks like for large scale sources

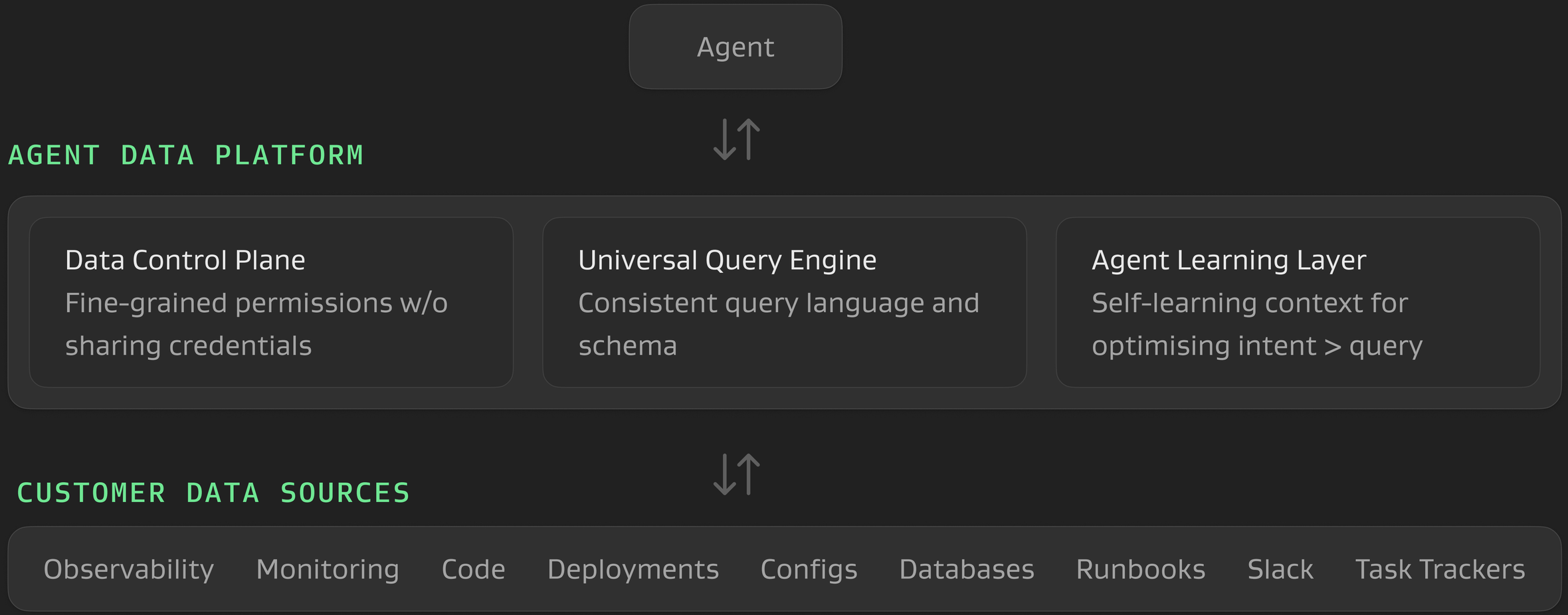
- ✓ **Precision over volume**
  Retrieve only what's relevant to the specific hypothesis — the right logs at the right granularity for the right time window.

- ✓ **Cross-source correlation at the query layer**
  Join signals before they reach the context window. Deployment timestamps correlated with error rate timelines — not left for the model.

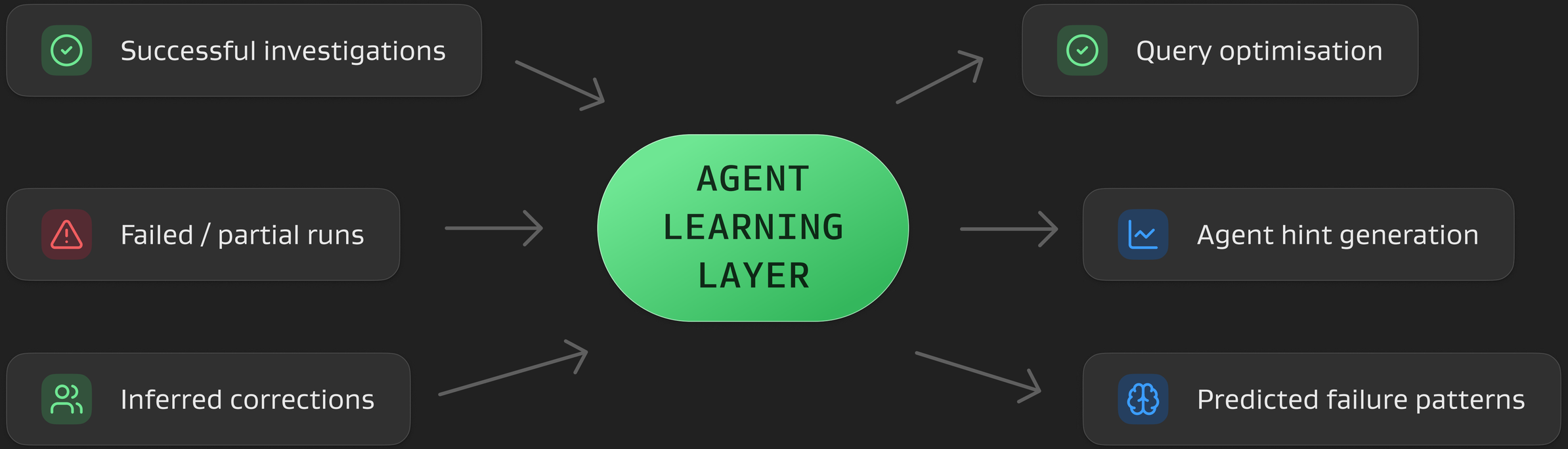
- ✓ **Schema abstraction**
  A familiar way for agents to understand how to navigate diverse schemas, whether its Prometheus, GitHub, PagerDuty — or whatever you have.

- ✓ **Cached system knowledge**
  Service topology, normal baselines, and historical failure patterns pre-loaded as context — not retrieved fresh each time.

How we improved data retrieval



Every run improves the derived data, making the agent more capable



1 Why software reliability should be more like an immune system

2 The importance of data retrieval

3 Which parts you should own in-house?

Five principles for trustworthy AI

- ✓ **Transparent reasoning**
  Structured hypotheses, evidence cited, confidence levels. No black-box outputs.
- ✓ **Strong guardrails and controls**
  Validations before actions. Clear boundaries. Graceful fallback when confidence is low.
- ✓ **A governed data layer**
  Precise, permissioned, cross-source retrieval. Every query auditable.
- ✓ **Continuous learning from every run**
  Successful, failed, and corrected investigations all feedback. The system compounds.
- ✓ **Graduated autonomy**
  Start with suggestions. Move to approved actions. Earn the right to act. Trust is built incrementally.

The Trap:
AI That Owns Your Intelligence

1. You connect your observability tools, code repos, and incident history to a SaaS AI product.
2. Their agents learn your system's behaviour over time, building a data model of failure patterns.
3. That accumulated intelligence — learned patterns, system context — lives in their cloud.
4. What happens when you need to switch vendors, renegotiate pricing, or meet a compliance requirement?
5. The vendor is holding the AI asset now, not you. Switching costs are higher because you don’t want to restart from scratch.

Retain control of your data and derived data



Your data never leaves your environment

Straightforward self-hosting with no data or API credentials exiting to a third-party.



The intelligence from derived data remains yours

The accumulated context of your system is in your infrastructure. You own it, inspect it, query it, or move it whenever you choose.



Every run compounds your advantage

Improvements derived from usage should be transparent and transferrable, not stuck within a vendor's product.

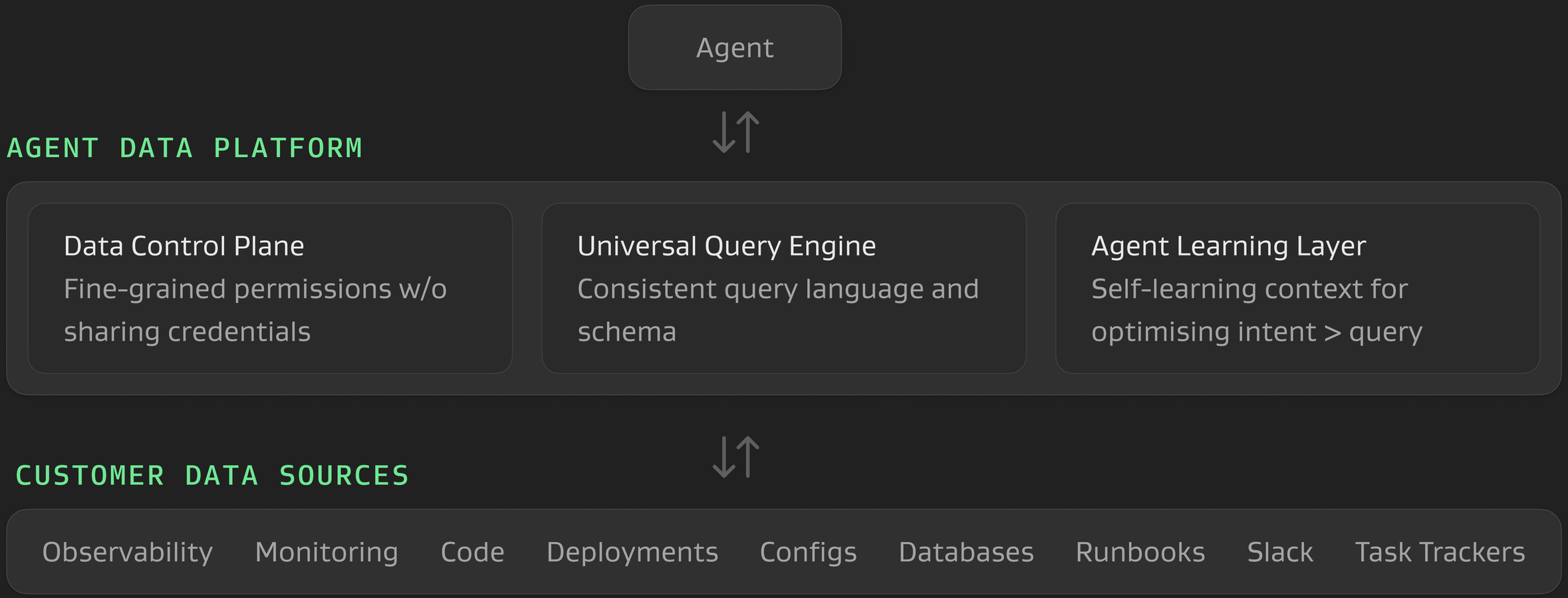


Switching costs collapse

The intelligence is data — a graph of entities, relationships and patterns — not just model weights trapped in an LLM.

The open source agent data platform

One connection for agents to safely and effectively query data



Use cases throughout the tech org

ALERT INVESTIGATION

Triage, hypothesis generation, evidence retrieval and proposed resolutions.

CODING AGENT CONTEXT

Identify latent weaknesses to mitigate and help coding agents avoid breaking changes.

INFRA OPTIMISATIONS

Identify idle or over-provisioned resources, spend anomalies and infra drift.

CUSTOMER ESCALATIONS

Disambiguating, triaging and diagnosing user technical escalations.

ICT COMPLIANCE

Demonstrate evidence of risk controls and automate incident reporting.

AND MORE…

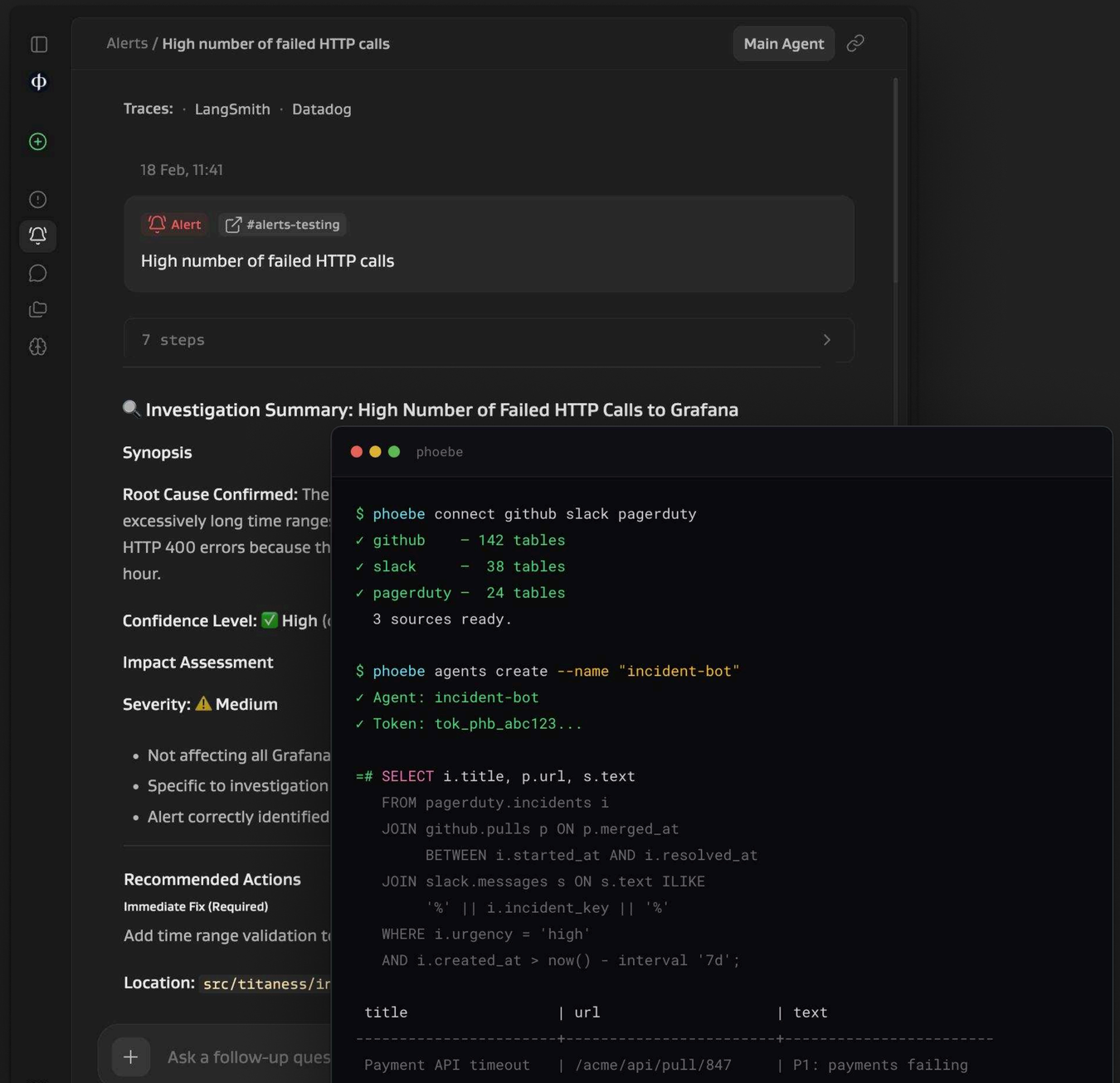
Wherever AI will benefit from secure access to fragmented or large scale operational data.

Built for agents and humans

Add your vendor SaaS or self-hosted data sources to Phoebe.

Your AI agents can query Phoebe via a single MCP (e.g. Cowork) or CLI (e.g. Claude Code, production agents).

For ad-hoc data queries, users can interact directly with Phoebe in the CLI, web UI or messaging apps.



Waitlist now open for the free, open source agent data platform

preview@phoebe.ai

- ✓ No data egress beyond your environment
- ✓ Connect to your existing data sources
- ✓ Bring your own LLM (OpenAI, Anthropic, local models)
- ✓ You own the intelligence